University of Roehampton London

LAST UPDATED: 20 September BY Sayed Pouria Talebi

SOFTWARE DEVELOPMENT 1 COURSEWORK SPECIFICATION

This course starts your software deployment journey by introducing the key programming concepts on which the rest of the course is built. The focus here is on a solid foundational knowledge of Python's syntax and semantics. To evaluate progress and highlight any gaps in understanding, this coursework will provide a set of challenging tasks and programming requirements, as well as ample time to reach an adequate solution to them.

Coursework Submission date: 10th December 2026 via Moodle by 17:00 hours GMT.

Aim: To gauge understanding on the correct use of variables and objects, data types and the ability to abstract data; as well as the develop students' fluency in programming languages and software development.

Learning Outcomes:

1- Design, implement, test, and debug a program that uses each of the following fundamental programming constructs, such as basic computation, simple I/O, standard conditional and iterative structures, the definition of functions, and parameter passing.
2- Analyse and explain the behaviour of simple programs involving the fundamental programming constructs, such as variables, expressions, assignments, I/O, control constructs, functions, parameter passing, and recursion.
3- Identify the relative strengths and weaknesses among multiple designs or implementations for a problem.
4- Use a programming language to implement, test, and debug algorithms for solving simple problems.

Grading Advice: While fully functioning programs and its documentation that meet all the requirements are the goal of the test, the programs and the rationale for their design, implementation, testing and debugging will be assessed for efficiency, simplicity, creativity and good style. Please read the Grading Rubric and bring any questions you may have to your tutor.

To complete this coursework, you will design the behavior of simple programs involving the fundamental programming constructs, e.g. variables, expressions, assignments, I/O, and control constructs.

PROBLEM DESCRIPTION

A university has hired our Software Development company to modernize their student assessment system. As we know, universities evaluate students based on various components throughout the academic year. There are several aspects to determining a student's overall performance, such as their attendance, coursework submissions, exam results, and participation. The university administration wishes to implement a categorical marking system to provide more detailed and standardized feedback to students. You have been tasked to create a program that automates this calculation and categorization process.

For the Software Development 1 module, student performance is assessed through the following components:

2. Coursework 1: Flowchart concepts (10%)

3. Coursework 2: Basic Programming concepts (20%)

3. Coursework 3: Complex program implementation (30%)

4. Test: Theoretical and practical assessment (40%)

You have been tasked to create a program that calculates a student's overall score for this module and assigns a category based on the university's categorical marking framework. The program should consider the weighted scores from each component, calculate the overall score, and then map it to the appropriate category:

- 100 (**Gold Standard**)
- 92, 85, 82 (**Upper First**)
- 78, 75, 72 (**First**)
- 68, 65, 62 (**Upper Second**)
- 58, 55, 52 (**Lower Second**)
- 48, 45, 42 (**Third**)
- 38, 35, 32 (**Condonable Fail**)
- 25, 15, 5 (**Fail**)
- 0 (**No Submission**) - nothing of merit, non-submission, or academic misconduct

Here's a basic outline of how the main calculation would work:
Calculate overall weighted score:
Overall_score = (coursework1 * 0.10)+ (coursework2 * 0.20) + (coursework3 * 0.30) + (final_exam * 0.40)
Then, select the appropriate categorical marking from this table:

No.	Category	Overall Mark
1	Gold Standard	100
2	Upper First	82, 85, 92
3	First	72, 75, 78
4	Upper Second	62, 65, 68
5	Lower Second	52, 55, 58
6	Third	42, 45, 48
7	Condonable Fail	32, 35, 38
8	Fail	5, 15, 25
9	No Submission	0

Example: If a student scores 58 in coursework 1, 65 in coursework 2, 55 in coursework 3, and 72 in the final exam, their overall score would be calculated as:

(58 * 0.10) + (65 * 0.20) + (55 * 0.30) + (72 * 0.40) = 5.8 + 13 + 16.5 + 28.8 = 64.1

This would be rounded to the nearest category, which is 65, placing the student in the "Upper Second" category.

You can find out more about categorical marking at:
https://moodle.roehampton.ac.uk/pluginfile.php/4731975/mod_resource/content/1/LTQC%2088.19%20Categorical%20Marking%20Frameworks%20UG%20PG%20.pdf

First Step

This step is not mandatory and has no grades associated. However, it is highly recommended to complete this task and discuss your results with your tutor in the Lab. You should be able to complete this task by the end of Week 3.

Blakedown the task to its constituent element and draw a plan on how you might implement a programme that asks for the required inputs to calculate the overall mark of a student, calculates the overall mark, and prints the result. You might like to draw a flowchart that you will use for your program design. You can write it down/draw it by hand and take a picture or choose any software you prefer.

Submission: Please submit a PDF file containing your flowchart on Moodle.

Second Step

This step is not mandatory and has no grades associated. However, it is highly recommended to complete this task and discuss your results with your tutor in the Lab. You should be able to complete this task by the end of Week 6.

There are two tasks you need to complete:

Task 1. Craft a program in Python which has the features listed below:

a. Your program should use the *input()* function that includes message text to instruct the user to enter each student's data.
b. Use an appropriate data structure to create and store this dataset, which contains the student's ID, and overall score.
c. Your program calculates the overall rate for each student, using the four main criteria described above (coursework 1, coursework 2, coursework 3 and final test)
d. Your program should estimate the suitable category for each student by using the category dataset provided. For instance, if the overall score of the student is between 72 and 78, the output should be: **First**
e. The computed score should be printed out as an integer (no decimals).

Task 2: Write a brief reflection on your code (~500 words), which may include but not limited to the discussions about topics below.

- How did you decompose the problem and why?
- Do you think flowchart helpful when you write a program and why?
- Have you met any difficulties/bugs during this coursework, and how did you tackle it?
- What did you think is the most difficult part in this coursework?
- What have you learnt from investigating this problem?
- Which part(s) in your code design is your favourite and why?
- If you had no time limit, how would you improve your program?

Resources on reflection can be found at: https://libguides.hull.ac.uk/reflectivewriting/reflection1a

Submission Requirements: Please submit your code as a **Python (*.py) or Jupyter Notebook (*.ipynb)** on Moodle. You can upload your reflection to the same link.

Feedback: Your code will be discussed with you in your lab. session.

COURSEWORK

This is your final submission and will determine your coursework mark.

To complete this coursework, you will design the behaviour of complex programs involving the fundamental programming constructs, such as functions, operations, and file reading/writing. A template Jupyter notebook and a text file are provided to complete the tasks. **Time in class can be used to troubleshoot and get feedback on your program.**

The university that hired our Software Development company is very pleased with the outcome of the first and second steps. Thus, they have requested an extension. If you recall, the program that you have crafted for the second step of this coursework takes in scores for 4 components of student assessment in the Computer Programming module:

1. Coursework 1: Flowchart concepts (10%)
2. Coursework 2: Basic Programming concepts (20%)
3. Coursework 3: Complex program implementation (30%)
4. Test: Theoretical and practical assessment (40%)

The program uses these individual component scores to calculate an overall score out of 100 using the formula:

Overall_score = (coursework1 * 0.10)+ (coursework2 * 0.20) + (coursework3 * 0.30) + (final_exam * 0.40)

The resulting number (should be between 0 and 100) is then used to estimate the category using the categorical marking dataset:

No.	Category	Overall Mark
1	Gold Standard	100
2	Upper First	82, 85, 92
3	First	72, 75, 78
4	Upper Second	62, 65, 68
5	Lower Second	52, 55, 58
6	Third	42, 45, 48
7	Condonable Fail	32, 35, 38
8	Fail	5, 15, 25
9	No Submission	0

From the dataset above,

- If the overall score is equal to 100, the output should be "Gold Standard".
- If the overall score is between 82 and 92, the output should be "Upper First".
- If the overall score is between 72 and 78, the output should be "First ".
- If the overall score is between 62 and 68, the output should be "Upper Second".
- If the overall score is between 52 and 58, the output should be "Lower Second".
- If the overall score is between 42 and 48, the output should be "Third”
- If the overall score is between 32 and 38, the output should be "Condonable Fail".
- If the overall score is between 5 and 25, the output should be "Fail".
- If the overall score is 0, the output should be "No Submission".

You are asked to add the following functionality:

1. Functions: Split and pack up your Coursework Midpoint Review code into functions. The program must contain a main() function and at least two other functions called "calculate_overall_score()" and "determine_category()".

2. Input expansion: Adjust your program to ask users to enter a student ID (2-digit number), name and date-of-birth (D.o.B) by sequence, before asking for the 4 component scores, i.e. Coursework1, Coursework 2, Coursework 3 and Final Exam. D.o.B should be in ISO format YEAR-MONTH-DAY.
3. Loop your code: Your program should use a loop to ask for the information of three students, or until the input for user ID is "end".
4. Calculate the students' overall scores and store them for later use.
5. Determine the students' categories and store them for later use.
6. Calculate students' ages (8 points): According to each user's year of birth, your program should calculate the age of the student and store it for later use. Hint, the library datetime can be used here to make your job easier.
7. Tabulate (8 points): Display a table that summarises the input data (ascending order with user ID) using the tabulate library. It should look like:

UID	Name	D.o.B	Age	Score	Category
13	Lieke Martens	1992-12-16	29	86.6667	Upper-First
02	Cristiano Ronaldo	1985-02-05	37	73.3333	First
24	Lucy Bronze	1991-10-28	31	50	Third

8. File writing (8 points): Save the table into a new local file named "students.txt"
9. Make sure that you:
    a. Implement robust input validation for a student's ID, D.o.B., and scores. Your program should give
    b. Warning that says "The input you entered was invalid" for any input errors (6 points)
    c. Display use of appropriate data structures for storing student information (5 points)
    d. Display use of appropriate code structures for implementing the functionality (5 points)
10. Implement a function called `round_to_category(score)` that takes a numerical score and rounds it to the nearest category. This function should (10 points):
    a. Accept a float or integer score as input.
    b. Determine the two nearest category boundaries.
    c. Round the score to the nearest category based on which boundary it's closer to.
    d. Return the rounded score and the corresponding category.

    For example:

    - If the input score is 73.7, it should return (75, "First") because 73.7 is closer to 75 than to 72.
    - If the input score is 69.2, it should return (68, "Upper Second") because 69.2 is closer to 68 than to 72.
    -Your implementation should handle edge cases, such as scores exactly halfway between two categories (round up in these cases), and scores at or beyond the extremes of the scale.

UID	Name	D.o.B	Age	Raw Score	Rounded Score	Category
13	Lieke Martens	1992-12-16	29	86.6667	85	Upper-First
02	Cristiano Ronaldo	1985-02-05	37	73.3333	75	First
24	Lucy Bronze	1991-10-28	31	50	52	Third

11. Distinction Advanced Function: Once everything in requirements 1-10 has been completed, for a distinction grade, extend your program with one more function called advanced(). This function should:
    a. Be a complementary main() method that, instead of taking user input, uses the provided file as input.
    b. Accepts in its argument (filename) the name of the text file to read.
    c. Read the student information record from the provided file "StudentData.txt", which contains student IDs, names, and date-of-births. The function should then use this data as the input for the program.
    d. Runs all the other functions used and produces a text file output (requirement 8).
12. Advanced Generalization Function (Distinction level): Implement a function called `setup_module()` that allows the user to define the assessment criteria for any module. This function should:
    a. Ask the user to input the module name.

b. Ask the user how many assessment components the module has.
c. For each component:
    - Ask for the component name (e.g., "Coursework 1", "Final Exam")
    - Ask for the component's weight as a percentage (ensure all weights sum to 100%)
d. Store this information in an appropriate data structure.
e. Return the module configuration.

Here's an example of how the program flow might look:

```
Welcome to the Student Grading System First, let's set up the module configuration.

Enter module name: Advanced Programming
How many assessment components does this module have? 3
Component 1 name: Midterm Exam
Component 1 weight (%): 30
Component 2 name: Final Project
Component 2 weight (%): 40
Component 3 name: Participation
Component 3 weight (%): 30
Module configuration complete.

Now, let's enter student information and grades.

Student 1:
ID: 01
Name: John Doe
Date of Birth (YYYY-MM-DD): 1995-05-15
Midterm Exam score: 75
Final Project score: 82
Participation score: 90

Results:
UID | Name     | D.o.B          | Age | Raw Score | Rounded Score | Category
-----|----------|----------------|------ |-----------|---------------------|---------
01 | John Doe | 1995-05-15 | 28   |82.6           | 82                  | Upper First
```

Submission Requirements: Submit your code via Moodle as a Jupyter notebook named:
student_category.ipynb

The program file should contain full documentation explaining your implementation (comment your code). Your file should have your name, ID number and date of last update.
Feedback: Feedback and will be provide in an in-person demo. You must book time for this demo form your lab or lecture touter. This can be done from the point that you submit till last working day in 18th of December of 2026. In the in-person demo you will have to demonstrate how your code meets the criteria in the Marking Scheme.

Marking Scheme: Grades will be based on number of components completed. You will be assessed on

- **Solution:** the completeness of the code, documentation, and reflection to meet the specification given.
- **Design:** ability to decompose a problem into coherent and reusable parts.
- **Correctness:** ability to create solution that reliably produces correct answers or appropriate results.
- **Logic:** ability to use correct program structures appropriate to the problem domain.

- **Clarity:** ability to format and document code for human consumption (PEP-8) and reflection.
- **Robustness:** ability of the solution to handle unexpected input and error conditions correctly as evidenced via testing.

The following rubric will be used to assess your work:

Basic Requirements
In order to get full marks for this rubric category your code must:
- Be encapsulated into functions including *`main()`,* `calculate_overall_score*()`,* and `determine_category*()`*. (8 points)
- Prompt users for an ID, name, D.o.B, and four student components, in that order. (8 points)
- Accept data for exactly 3 students before stopping or if "end" was entered instead of an ID. (8 points)
- Calculate the student's overall score. (2 points)
- Calculate the student's category. (2 points)
- Calculate the students' ages. (8 points)
- Display a table of all the data passed to the program using `tabulate` (8 points)
- Save this table to `students.txt` (8 points)

Intermediate requirements
To get full marks for the rubric category your code must:
- Implement robust input validation for a student's ID, D.o.B., and overall score (6 points)
- Display use of appropriate data structures for storing student information (5 points)
- Display use of appropriate code structures for implementing the functionality (5 points)

Distinction Requirements:
Distinction work in the range of 72-78 should demonstrate:
- Exemplary attainment of learning outcomes
- A high level of insight and critical evaluation of the material
- A comprehensive and up-to-date account of relevant theoretical and empirical material
- A thorough understanding and integration of material supporting a cogent argument
- Excellent writing of a high academic standard

Distinction work in the range of 82-100 should demonstrate:
- Attainment beyond the intended learning outcomes
- An outstanding level of originality and creativity, providing a significant new perspective on the question or topic
- A clear, elegant and well supported argument, based on the integration and sophisticated critical evaluation of a substantial body of knowledge
- Suitability for publication in high quality journal

Academic Misconduct: All submissions will be processed through a code plagiarism tool. If signs of misconduct are found, all students involved will be contacted to discuss further steps. Please see here for information on academic integrity at the university https://portal.roehampton.ac.uk/information/Pages/Academic-Integrity.aspx.
Our guiding principle is that academic integrity and honesty are fundamental to the academic work you produce at the University of Roehampton. You are expected to complete coursework which is your own and which is referenced appropriately. The university has in place measures to detect academic dishonesty in all its forms. If you are found to be cheating or attempting to gain an unfair advantage over other students in any way, this is considered academic misconduct and you will be penalised accordingly. Please don't do it.

Marking Scheme:

A short video explaining the assessment criteria can be found at: https://www.youtube.com/watch?v=JrA0Dotq1p8

The following rubric will be used to assess your work:

Criteria	Excellent	Satisfactory	Not Satisfactory	Not Attempted
Solution the completeness of the code, documentation and reflection to meet the specification given.	A completed solution meeting all the specifications.	A completed solution that does not quite meet all the specifications.	A completed solution is implemented and runs but lacks much of the specification.	Solution doesn't run or does not meet the specifications defined.
Design ability to decompose a problem into coherent and reusable parts.	The solution design decomposes the problem by using appropriate structures. The overall design is appropriate, and with a consideration of reusability.	The solution design generally uses appropriate structures. Program elements and documentation both exhibit good design.	Not all of the selected structures are appropriate. Some of the elements are appropriately designed.	Few if any of the selected structures are appropriate, or the problem is decomposed in ways that make little sense.
Correctness ability to create solution that reliably produces correct answers or appropriate results.	Solution produces correct answers or appropriate results for all inputs tested.	Solution produces correct answers or appropriate results for most inputs.	Solution approaches correct answers or appropriate results for most inputs but can contain miscalculations.	Solution does not produce correct answers or appropriate results for most inputs.
Logic ability to use correct program structures appropriate to the problem domain.	Solution logic is correct with no known boundary errors, and no redundant or contradictory conditions.	Solution logic is mostly correct but may contain an occasional boundary error or redundant or contradictory condition.	Solution logic is on the right track but shows no recognition of boundary conditions (such as < vs. <=).	Solution contains some conditions that specify the opposite of what is required (less than vs. greater than), confuse Boolean AND/OR operators, or lead to infinite loops.
Clarity ability to format and document code for human consumption (Good Style) and reflection.	Program contains appropriate documentation for all major functions, variables, or non-trivial algorithms. Formatting, indentation, and other white space aids readability.	Program contains some documentation on major functions, variables, or non-trivial algorithms. Indentation and other formatting are appropriate.	Program contains some documentation (at least the student's name and program's purpose) but has occasionally misleading indentation.	Program contains no documentation, or grossly misleading indentation.

Robustness ability of the solution to handle unexpected input and error conditions correctly as evidenced via testing.	Solution handles erroneous or unexpected input gracefully; action is taken without surprising the user. Boundary cases are considered and tested.	All obvious error conditions are checked for and appropriate action is taken. Nearly all boundary cases are considered and tested.	Some obvious error conditions are checked for and some sort of action is taken. Most boundary cases are considered and tested.	Solution often fails or fails completely. Boundary conditions are not tested for.